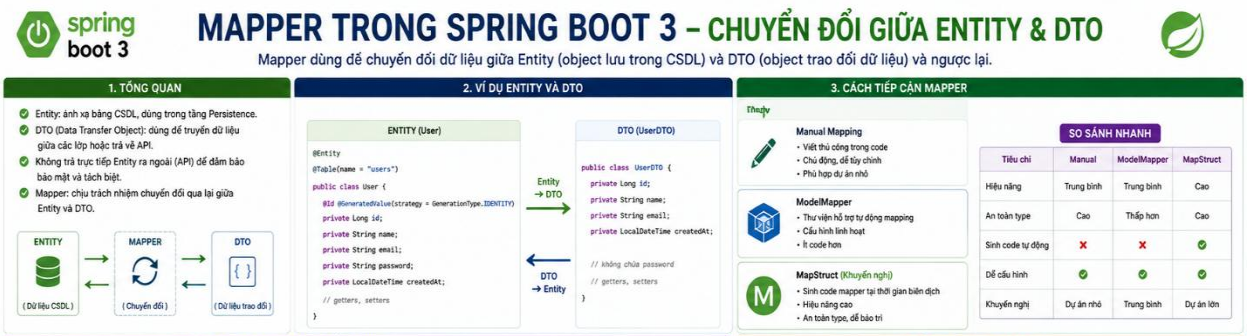
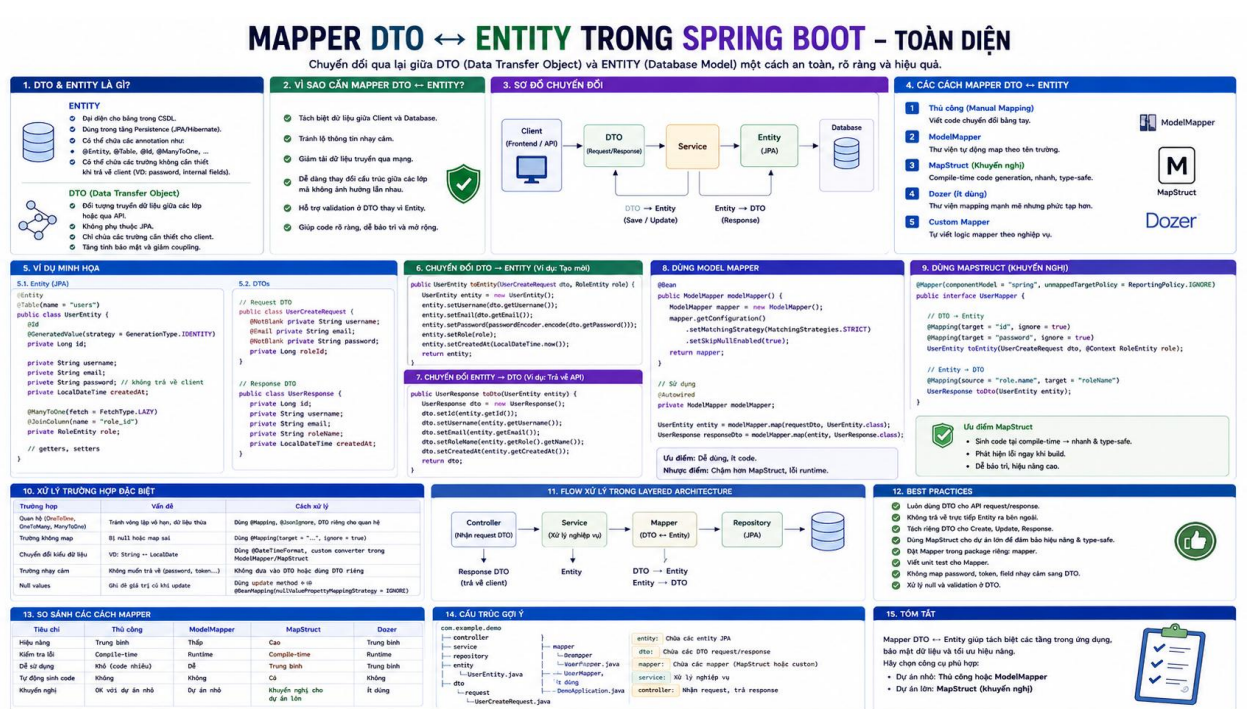
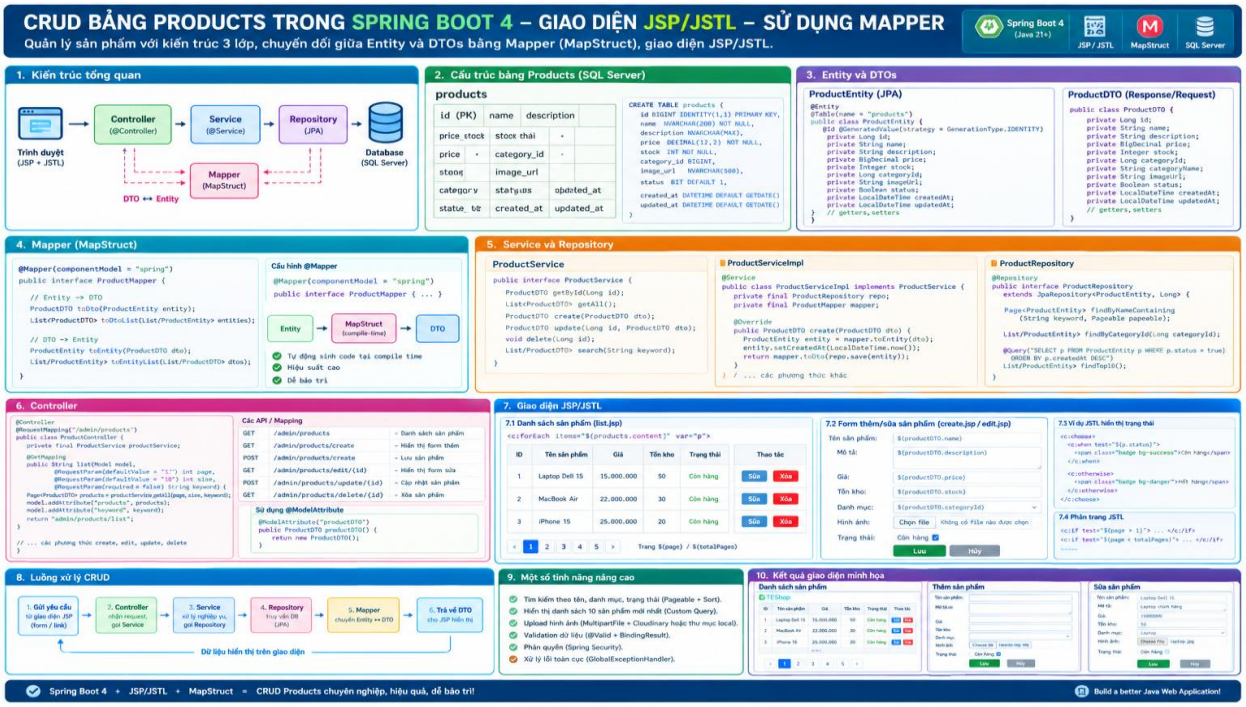
HƯỚNG DẪN MAPPER DTO TRONG SPRING BOOT 4

1. KIẾN THỨC







Mapper thủ công:

```
package vn.iotstar.mapper;

import org.springframework.stereotype.Component;

import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

import lombok.RequiredArgsConstructor;


import java.util.stream.Collectors;

@Component
@RequiredArgsConstructor
public class ProductMapper {

    private final ProductImageMapper imageMapper;

    // =====================================
    // ENTITY → DTO
    // =====================================

    public ProductDTO toDTO(Product entity) {

        if (entity == null) {
            return null;
        }

        return ProductDTO.builder()
                .id(entity.getId())
                .name(entity.getName())
                .price(entity.getPrice())
```

```
                .quantity(entity.getQuantity())
                .description(entity.getDescription())

                .images(
                    entity.getImages()
                        .stream()
                        .map(imageMapper::toDTO)
                        .collect(
                            Collectors.toList()
                        )
                )

                .build();
    }

    // ====================================
    // DTO → ENTITY
    // ====================================

    public Product toEntity(
            ProductDTO dto) {

        if (dto == null) {
            return null;
        }

        return Product.builder()
                .id(dto.getId())
                .name(dto.getName())
                .price(dto.getPrice())
                .quantity(dto.getQuantity())
                .description(dto.getDescription())
                .build();
    }

    // ====================================
    // UPDATE
    // ====================================

    public void updateEntity(
            ProductDTO dto,
            Product entity) {

        entity.setName(dto.getName());
        entity.setPrice(dto.getPrice());
        entity.setQuantity(dto.getQuantity());
        entity.setDescription(
                dto.getDescription()
        );
    }
}
```

Mapper dùng Mapstruct:

Bước 1: Thêm thư viện vào file pom.xml

```
<dependency>
                    <groupId>org.mapstruct</groupId>
```

```
                <artifactId>mapstruct</artifactId>
                <version1.6.3</version>
            </dependency>
            <dependency>
                <groupId>org.mapstruct</groupId>
                <artifactId>mapstruct-processor</artifactId>
                <version1.6.3</version>
                <scope>provided</scope>
            </dependency>
```

Bước 2: Thêm cấu hình mastruct trong phần <build> của file pom.xml

```
<build>
            <plugins>
                <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-compiler-plugin</artifactId>
                    <configuration>

               <encoding>UTF-8</encoding>

                            <annotationProcessorPaths>
                                <path>
                                    <groupId>org.mapstruct</groupId>
                                    <artifactId>mapstruct-processor</artifactId>
                                    <version>${mapstruct.version}</version>
                                </path>
                                <path>
                                    <groupId>org.projectlombok</groupId>
                                    <artifactId>lombok</artifactId>
                                </path>
                            </annotationProcessorPaths>
                    </configuration>
                </plugin>
                <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
                </plugin>
            </plugins>
        </build>
```

Bước 3: Tạo class mapper

```
package vn.iotstar.mapper;
import org.mapstruct.*;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

@Mapper(componentModel = "spring", unmappedTargetPolicy = ReportingPolicy.IGNORE)
public interface ProductMapper {
    @Mapping(target = "userId", source = "user.id")
    @Mapping(target = "username", source = "user.username")
    @Mapping(target = "image", ignore = true)
    ProductDTO toDTO(Product entity);

    @Mapping(target = "user", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    Product toEntity(ProductDTO dto);
}
```

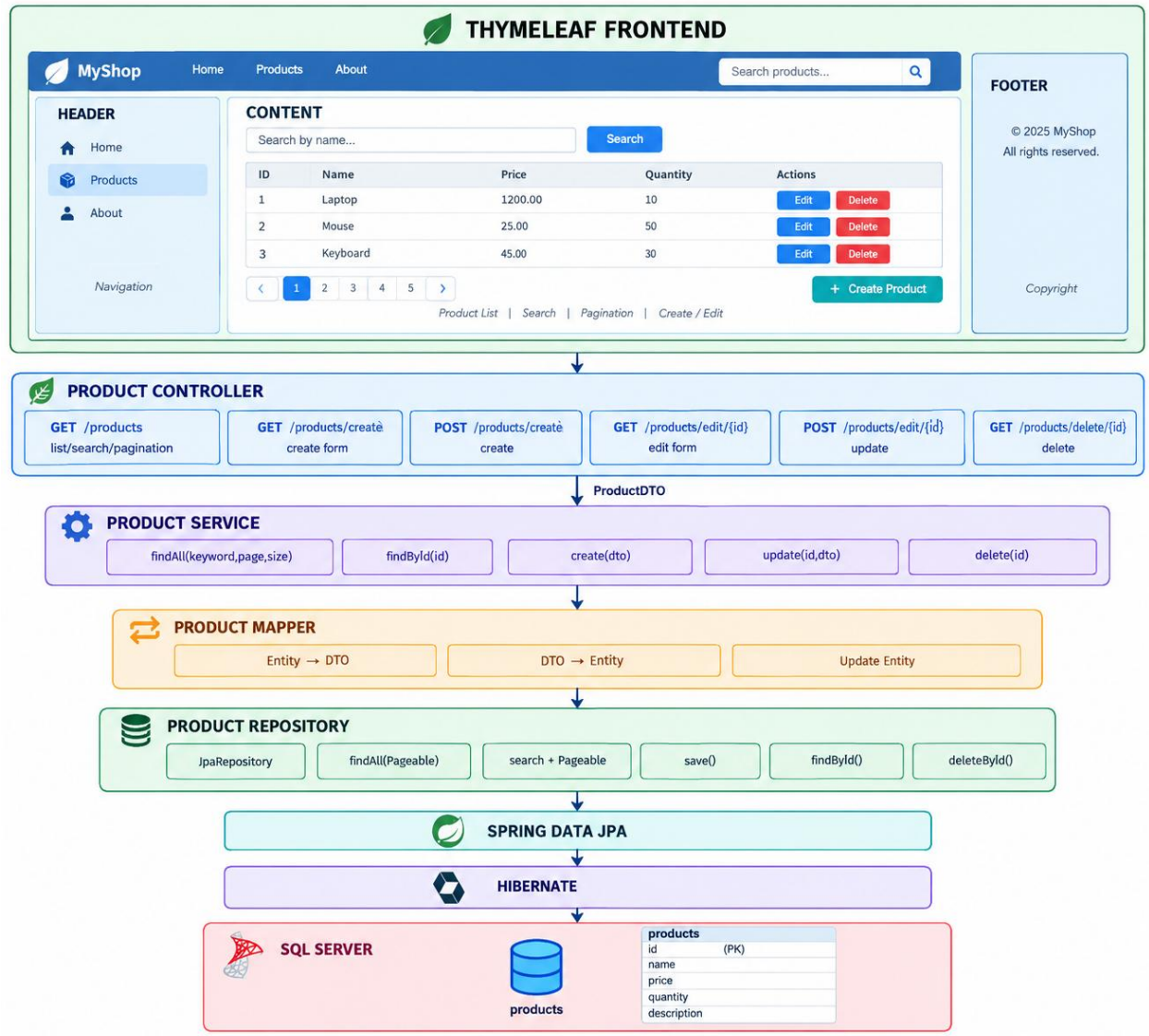
```java
package vn.iotstar.mapper;

import org.mapstruct.*;
import vn.iotstar.dto.UserDTO;
import vn.iotstar.entity.User;

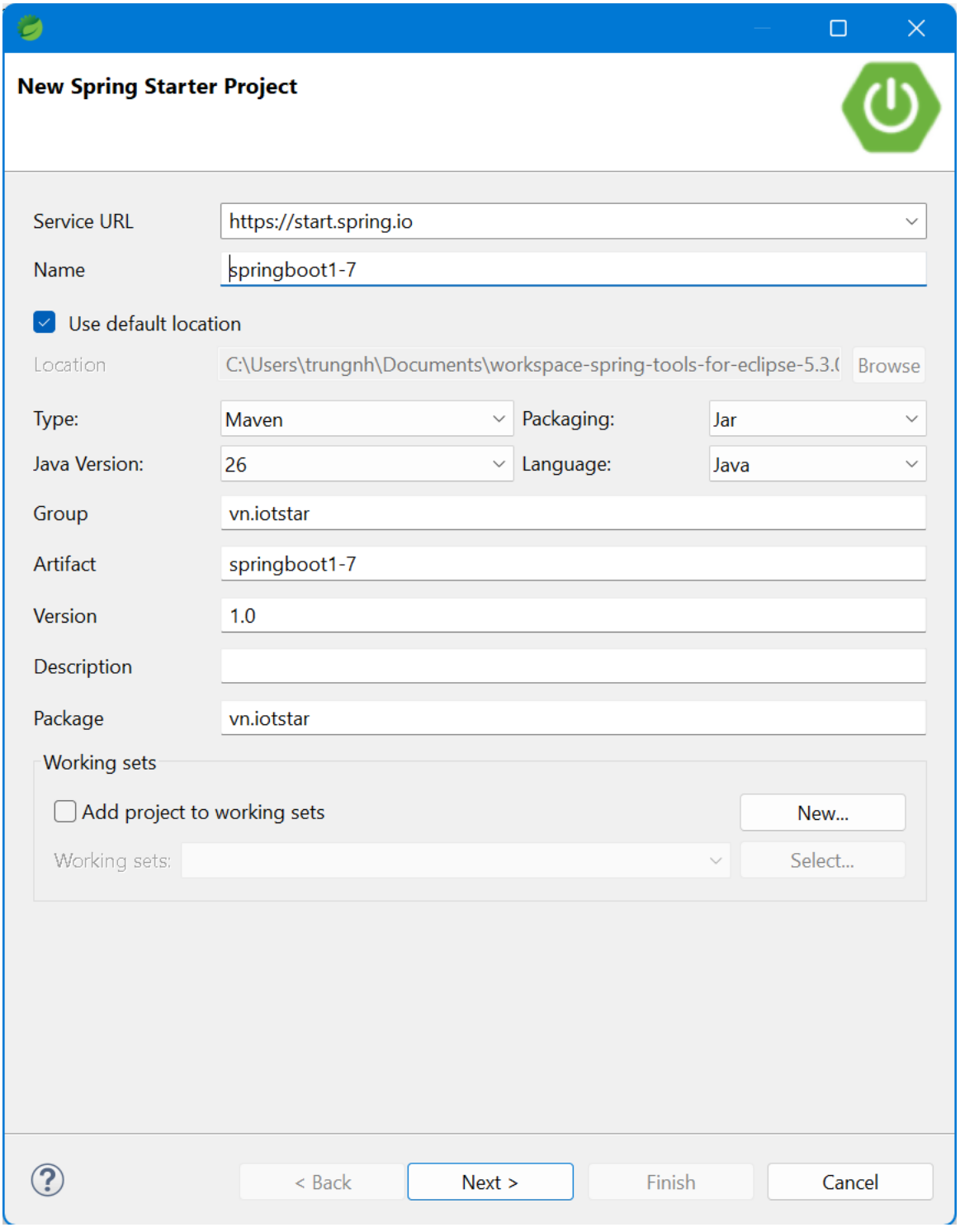
@Mapper(componentModel = "spring",
        unmappedTargetPolicy = ReportingPolicy.IGNORE)
public interface UserMapper {
    @Mapping(target = "roleName", source = "role.name")
    UserDTO toDTO(User entity);

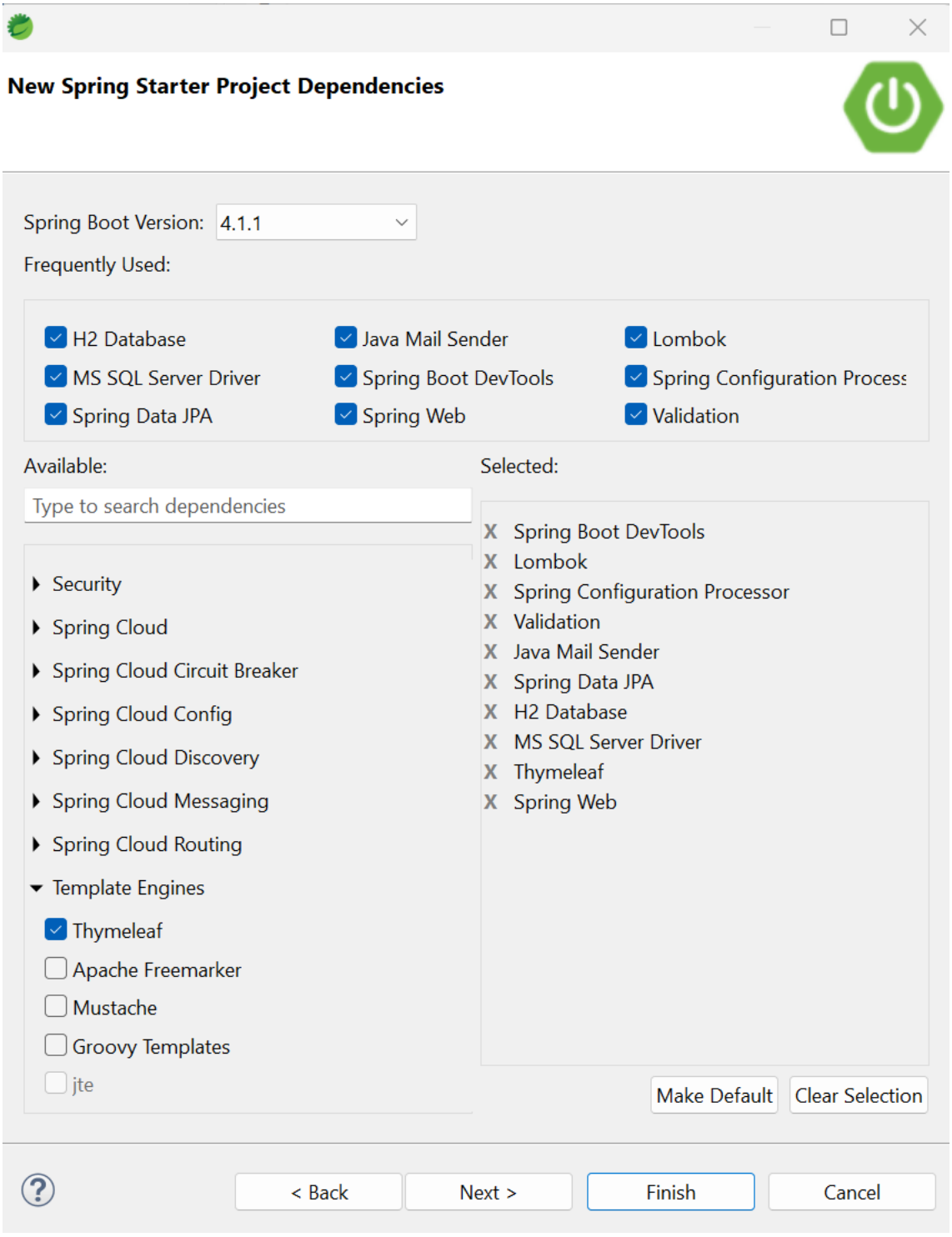
    @Mapping(target = "role", ignore = true)
    @Mapping(target = "products", ignore = true)
    @Mapping(target = "password", ignore = true)
    User toEntity(UserDTO dto);
}
```

2. VÍ DỤ: Xây dựng chức năng CRUD có tìm kiếm, phân trang cho bảng Product



Bước 1: Tạo project spring boot 4 với các thư viện sau:


New Spring Starter Project
Service URL
https://start.spring.io
Name
springboot1-7
Use default location
Location
C:\Users\trungnh\Documents\workspace-spring-tools-for-eclipse-5.3.(
Browse
Type:
Maven
Packaging:
Jar
Java Version:
26
Language:
Java
Group
vn.iotstar
Artifact
springboot1-7
Version
1.0
Description
Package
vn.iotstar
Working sets
Add project to working sets
New...
Working sets:
Select...
< Back
Next >
Finish
Cancel



Ngoài các thư viện trên, chúng ta thêm thư viện layout thymeleaf dialect sau:

```
<dependency>
	<groupId>nz.net.ultraq.thymeleaf</groupId>
	<artifactId>thymeleaf-layout-dialect</artifactId>
	<scope>compile</scope>
</dependency>
```

Bước 2: Mở file application.properties lên cấu hình

```
server.port=8099

# Thymeleaf
#spring.thymeleaf.prefix=classpath:/templates/
#spring.thymeleaf.suffix=.html
spring.thymeleaf.cache=false


#kết nối database
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=webst2;encrypt=false;tr
ustServerCertificate=true;sslProtocol=TLSv1.2;characterEncoding=UTF-8
spring.datasource.username=sa
spring.datasource.password=1234567@a$
spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver

#JPA
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
#Hibernate ddl auto (create, create-drop, update, none)
spring.jpa.hibernate.ddl-auto=update

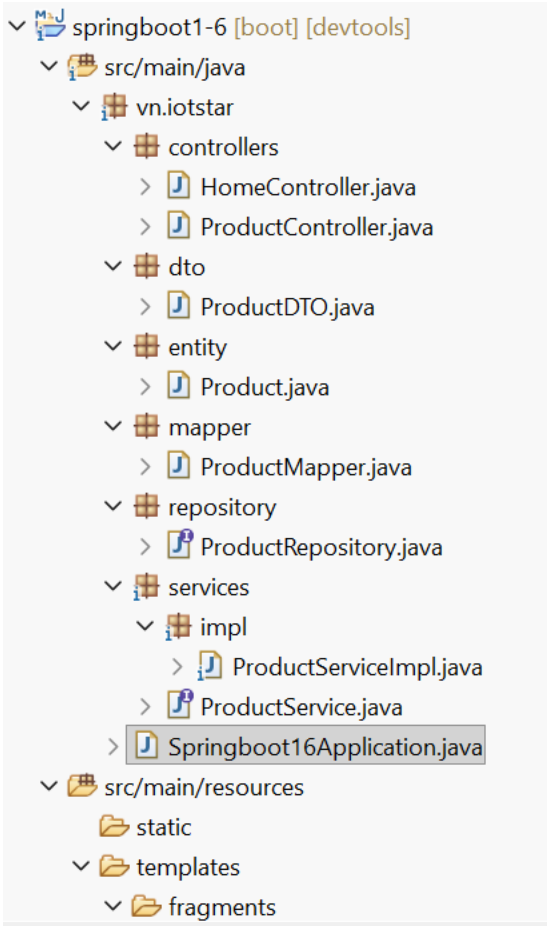
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
```

Bước 3: Cấu hình UTF-8 cho spring boot trong file SpringBootApplication.java

```
@Bean
	@Order(Ordered.HIGHEST_PRECEDENCE)
	 CharacterEncodingFilter characterEncodingFilter() {
	    CharacterEncodingFilter filter = new CharacterEncodingFilter();
	    filter.setEncoding("UTF-8");
	    filter.setForceEncoding(true);
	    return filter;
	}
```

Thêm @Configuration trên class để khai báo cho IOC container.

Bước 4: Tạo entity



Product.java:

```java
package vn.iotstar.entity;

import java.math.BigDecimal;
import java.time.LocalDateTime;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200, columnDefinition = "NVARCHAR(1000)")
    private String name;

    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal price;

    @Column(nullable = false)
    private Integer quantity;

    @Column(length = 1000, columnDefinition = "NVARCHAR(1000)")
    private String description;

    @Column(name = "created_at")
    private LocalDateTime createdAt;
}
```

Bước 5: Tạo ProductDTO, Không nên đưa trực tiếp Entity vào View. Sử dụng: Entity ⇄ Mapper ⇄ DTO

```
package vn.iotstar.dto;

import java.math.BigDecimal;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductDTO {

    private Long id;

    @NotBlank(message = "Tên sản phẩm không được để trống")
    @Size(max = 200, message = "Tên sản phẩm tối đa 200 ký tự")
    private String name;

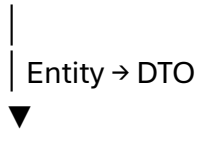
    @NotNull(message = "Giá không được để trống")
    @DecimalMin(value = "0", message = "Giá phải >= 0")
    private BigDecimal price;

    @NotNull(message = "Số lượng không được để trống")
    @Min(value = 0, message = "Số lượng phải >= 0")
    private Integer quantity;
```

```
    @Size(max = 1000)
    private String description;
}
```

Bước 6: Tạo ProductMapper, Mapper chịu trách nhiệm:

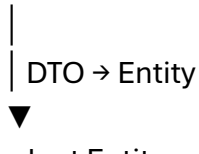
Product Entity



ProductDTO

và:

ProductDTO



Product Entity

```
package vn.iotstar.mapper;

import org.springframework.stereotype.Component;

import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

@Component
public class ProductMapper {

    public ProductDTO toDTO(Product entity) {

        if (entity == null) {
            return null;
        }

        return ProductDTO.builder()
                .id(entity.getId())
                .name(entity.getName())
                .price(entity.getPrice())
                .quantity(entity.getQuantity())
                .description(entity.getDescription())
                .build();
    }

    public Product toEntity(ProductDTO dto) {

        if (dto == null) {
            return null;
        }

        return Product.builder()
                .id(dto.getId())
                .name(dto.getName())
                .price(dto.getPrice())
                .quantity(dto.getQuantity())
```

```
                .description(dto.getDescription())
                .build();
    }

    public void updateEntity(ProductDTO dto,
                             Product entity) {

        entity.setName(dto.getName());
        entity.setPrice(dto.getPrice());
        entity.setQuantity(dto.getQuantity());
        entity.setDescription(dto.getDescription());
    }
}
```

Điểm quan trọng là khi **Update**, không nhất thiết tạo Entity mới.

ProductDTO



ProductMapper.updateEntity()



Product đang tồn tại



Repository.save()

Bước 7: tạo ProductRepository, Spring Data JPA hỗ trợ Pageable, Khi người dùng tìm:

GET /products?keyword=laptop&page=0&size=5 thì JPA sẽ thực hiện truy vấn phân trang

```
package vn.iotstar.repository;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

import vn.iotstar.entity.Product;

public interface ProductRepository extends JpaRepository<Product, Long> {


    Page<Product> findByNameContainingIgnoreCase(String name, Pageable pageable);
}
```

Bước 8: Tạo ProductService

```
package vn.iotstar.services;

import org.springframework.data.domain.Page;

import vn.iotstar.dto.ProductDTO;

public interface ProductService {

    Page<ProductDTO> findAll(String keyword, int page, int size);
```

```
        ProductDTO findById(Long id);

        ProductDTO create(ProductDTO dto);

        ProductDTO update(Long id, ProductDTO dto);

        void delete(Long id);
}
```

Bước 9: tạo ProductServiceImpl

```
package vn.iotstar.services.impl;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import lombok.RequiredArgsConstructor;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;
import vn.iotstar.mapper.ProductMapper;
import vn.iotstar.repository.ProductRepository;
import vn.iotstar.services.ProductService;

@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService {

    private final ProductRepository productRepository;
    private final ProductMapper productMapper;

    @Override
    @Transactional(readOnly = true)
    public Page<ProductDTO> findAll(
            String keyword,
            int page,
            int size) {

        Pageable pageable =
                PageRequest.of(
                        page,
                        size,
                        Sort.by("id").descending()
                );

        Page<Product> products;

        if (keyword == null || keyword.isBlank()) {

            products = productRepository
                    .findAll(pageable);

        } else {
```

```
        products = productRepository
                .findByNameContainingIgnoreCase(
                        keyword.trim(),
                        pageable
                );
    }

    return products.map(productMapper::toDTO);
}

@Override
@Transactional(readOnly = true)
public ProductDTO findById(Long id) {

    Product product = productRepository
            .findById(id)
            .orElseThrow(() ->
                    new RuntimeException(
                            "Không tìm thấy sản phẩm: " + id
                    )
            );

    return productMapper.toDTO(product);
}

@Override
public ProductDTO create(ProductDTO dto) {

    Product product =
            productMapper.toEntity(dto);

    Product saved =
            productRepository.save(product);

    return productMapper.toDTO(saved);
}

@Override
public ProductDTO update(
        Long id,
        ProductDTO dto) {

    Product product =
            productRepository
                    .findById(id)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy sản phẩm"
                            )
                    );

    productMapper.updateEntity(dto, product);

    Product updated =
            productRepository.save(product);

    return productMapper.toDTO(updated);
```

```
    }

    @Override
    public void delete(Long id) {

        if (!productRepository.existsById(id)) {
            throw new RuntimeException(
                    "Không tìm thấy sản phẩm"
            );
        }

        productRepository.deleteById(id);
    }
}
```

Bước 10: Tạo ProductController

```
package vn.iotstar.controllers;

import jakarta.validation.Valid;

import lombok.RequiredArgsConstructor;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.services.ProductService;

import org.springframework.data.domain.Page;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    // =========================
    // LIST + SEARCH + PAGINATION
    // =========================

    @GetMapping
    public String listProducts(

            @RequestParam(
                    defaultValue = ""
            )
            String keyword,

            @RequestParam(
                    defaultValue = "0"
            )
            int page,

            @RequestParam(
                    defaultValue = "5"
            )
```

```
        int size,

        Model model) {

    Page<ProductDTO> productPage =
            productService.findAll(
                    keyword,
                    page,
                    size
            );

    model.addAttribute(
            "products",
            productPage
    );

    model.addAttribute(
            "keyword",
            keyword
    );

    model.addAttribute(
            "size",
            size
    );

    return "products/list";
}

// ========================
// CREATE FORM
// ========================

@GetMapping("/create")
public String createForm(Model model) {

    model.addAttribute(
            "product",
            new ProductDTO()
    );

    model.addAttribute(
            "formTitle",
            "Thêm sản phẩm"
    );

    return "products/form";
}

// ========================
// CREATE
// ========================

@PostMapping("/create")
public String create(

        @Valid
        @ModelAttribute("product")
```

```
        ProductDTO dto,

        BindingResult result) {

    if (result.hasErrors()) {
        return "products/form";
    }

    productService.create(dto);

    return "redirect:/products";
}

// =========================
// EDIT FORM
// =========================

@GetMapping("/edit/{id}")
public String editForm(
        @PathVariable Long id,
        Model model) {

    ProductDTO product =
            productService.findById(id);

    model.addAttribute(
            "product",
            product
    );

    model.addAttribute(
            "formTitle",
            "Cập nhật sản phẩm"
    );

    return "products/form";
}

// =========================
// UPDATE
// =========================

@PostMapping("/edit/{id}")
public String update(

        @PathVariable Long id,

        @Valid
        @ModelAttribute("product")
        ProductDTO dto,

        BindingResult result) {

    if (result.hasErrors()) {
        return "products/form";
    }

    productService.update(id, dto);
```

```
        return "redirect:/products";
    }

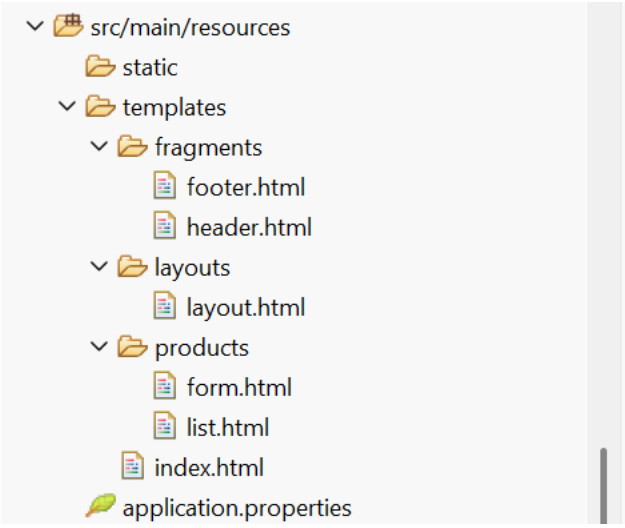
    // =========================
    // DELETE
    // =========================

    @GetMapping("/delete/{id}")
    public String delete(
            @PathVariable Long id) {

        productService.delete(id);

        return "redirect:/products";
    }
}
```

Bước 11: Tạo view bằng thymeleaf



layouts/layout.html

```
<!DOCTYPE html>

<html lang="vi"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">

<head>

    <meta charset="UTF-8">

    <title layout:title-pattern="$CONTENT_TITLE - Product Manager">
        Product Manager
    </title>

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">

</head>
```

```
<body>

    <!-- HEADER -->
    <div th:replace="~{fragments/header :: header}">
    </div>


    <!-- CONTENT -->

    <main class="container mt-4"
          layout:fragment="content">

    </main>


    <!-- FOOTER -->

    <div th:replace="~{fragments/footer :: footer}">
    </div>

</body>

</html>
```

---

fragments/header.html

```
<!DOCTYPE html>

<html
    xmlns:th="http://www.thymeleaf.org">

<body>

<header th:fragment="header">

    <nav class="navbar navbar-dark bg-dark">

        <div class="container">

            <a class="navbar-brand"
               th:href="@{/products}">
                Product Management
            </a>

            <div>

                <a
                    class="btn btn-outline-light"
                    th:href="@{/products}">
                    Products
                </a>

            </div>

        </div>

    </nav>
```

```
</header>

</body>

</html>
```

Footer.html

```
<!DOCTYPE html>

<html
    xmlns:th="http://www.thymeleaf.org">

<body>

<footer
    th:fragment="footer"
    class="text-center mt-5 p-4 bg-light">

    <p>
        Spring Boot 4 - Product Management
    </p>

</footer>

</body>

</html>
```

---

products/list.html

```
<!DOCTYPE html>

<html lang="vi"

      xmlns:th="https://www.thymeleaf.org"

      xmlns:layout=
      "http://www.ultraq.net.nz/thymeleaf/layout"

      layout:decorate="~{layouts/layout}">

<head>

    <title>Danh sách sản phẩm</title>

</head>

<body>

<section layout:fragment="content">

    <div class="d-flex justify-content-between">

        <h2>
```

```
        Danh sách sản phẩm
    </h2>

    <a
        class="btn btn-primary"
        th:href="@{/products/create}">

        + Thêm sản phẩm

    </a>

</div>


<!-- SEARCH -->

<form
    class="row mt-3 mb-3"
    method="get"
    th:action="@{/products}">

    <div class="col-md-6">

        <input
            type="text"
            class="form-control"
            name="keyword"
            th:value="${keyword}"
            placeholder="Nhập tên sản phẩm...">

    </div>

    <div class="col-md-2">

        <select
            class="form-select"
            name="size">

            <option value="5"
                    th:selected="${size == 5}">
                5
            </option>

            <option value="10"
                    th:selected="${size == 10}">
                10
            </option>

            <option value="20"
                    th:selected="${size == 20}">
                20
            </option>

        </select>

    </div>

    <div class="col-md-2">
```

```
        <button
            class="btn btn-success"
            type="submit">

            Tìm kiếm

        </button>

    </div>

</form>


<!-- TABLE -->

<table
    class="table table-bordered table-hover">

    <thead class="table-dark">

    <tr>

        <th>ID</th>

        <th>Tên sản phẩm</th>

        <th>Giá</th>

        <th>Số lượng</th>

        <th>Mô tả</th>

        <th>Thao tác</th>

    </tr>

    </thead>


    <tbody>

    <tr
        th:each="product : ${products.content}">

        <td
            th:text="${product.id}">
        </td>

        <td
            th:text="${product.name}">
        </td>

        <td>

            <span
                th:text="${#numbers.formatDecimal(
                    product.price, 0, 'COMMA', 2, 'POINT'
```

```
                )}">
            </span>

        </td>

        <td
            th:text="${product.quantity}">
        </td>

        <td
            th:text="${product.description}">
        </td>

        <td>

            <a
                class="btn btn-warning btn-sm"
                th:href="@{/products/edit/{id}(id=${product.id})}">

                Sửa

            </a>

            <a
                class="btn btn-danger btn-sm"
                th:href="@{/products/delete/{id}(id=${product.id})}"
                onclick="return confirm('Bạn có chắc muốn xóa?')">

                Xóa

            </a>

        </td>

    </tr>

    </tbody>

</table>


<!-- EMPTY -->

<div
    class="alert alert-warning"
    th:if="${products.totalElements == 0}">

    Không tìm thấy sản phẩm.

</div>


<!-- PAGINATION -->

<nav
    th:if="${products.totalPages > 1}">

    <ul class="pagination">
```

```
<!-- Previous -->

<li
    class="page-item"
    th:classappend="${products.first} ? 'disabled'">

    <a
        class="page-link"

        th:href="@{/products(
            page=${products.number - 1},
            size=${size},
            keyword=${keyword}
        )}">

        Previous

    </a>

</li>


<!-- Page number -->

<li
    class="page-item"

    th:each="i : ${#numbers.sequence(
        0,
        products.totalPages - 1
    )}"

    th:classappend="${i == products.number}
        ? 'active'">

    <a
        class="page-link"

        th:text="${i + 1}"

        th:href="@{/products(
            page=${i},
            size=${size},
            keyword=${keyword}
        )}">

    </a>

</li>


<!-- Next -->

<li
    class="page-item"

    th:classappend="${products.last}
```

```
                    ? 'disabled'">

                <a
                    class="page-link"

                    th:href="@{/products(
                        page=${products.number + 1},
                        size=${size},
                        keyword=${keyword}
                    )}">

                    Next

                </a>

            </li>

        </ul>

    </nav>

</section>

</body>

</html>
```

products/form.html

```
<!DOCTYPE html>

<html lang="vi"

      xmlns:th="http://www.thymeleaf.org"

      xmlns:layout=
      "http://www.ultraq.net.nz/thymeleaf/layout"

      layout:decorate="~{layouts/layout}">

<head>

    <title th:text="${formTitle}">
        Product
    </title>

</head>

<body>

<section layout:fragment="content">

    <h2 th:text="${formTitle}">
        Product
    </h2>


    <form
```

```
method="post"

th:object="${product}"

th:action="${product.id == null}
    ? @{/products/create}
    : @{/products/edit/{id}(id=${product.id})}">


<!-- ID -->

<input
    type="hidden"
    th:field="*{id}">


<!-- NAME -->

<div class="mb-3">

    <label>
        Tên sản phẩm
    </label>

    <input
        type="text"
        class="form-control"
        th:field="*{name}">

    <div
        class="text-danger"
        th:errors="*{name}">
    </div>

</div>


<!-- PRICE -->

<div class="mb-3">

    <label>
        Giá
    </label>

    <input
        type="number"
        step="0.01"
        class="form-control"
        th:field="*{price}">

    <div
        class="text-danger"
        th:errors="*{price}">
    </div>

</div>
```

```
    <!-- QUANTITY -->

    <div class="mb-3">

        <label>
            Số lượng
        </label>

        <input
            type="number"
            class="form-control"
            th:field="*{quantity}">

        <div
            class="text-danger"
            th:errors="*{quantity}">
        </div>

    </div>


    <!-- DESCRIPTION -->

    <div class="mb-3">

        <label>
            Mô tả
        </label>

        <textarea
            class="form-control"
            rows="5"
            th:field="*{description}">
        </textarea>

    </div>


    <button
        type="submit"
        class="btn btn-primary">

        Lưu

    </button>


    <a
        th:href="@{/products}"
        class="btn btn-secondary">

        Quay lại

    </a>

</form>
```

```
</section>

</body>

</html>
```

Ví dụ 2: hêm thuộc tính images vào bảng products và điều chỉnh code để có thể upload và hiển thị images trên thư mục của project.

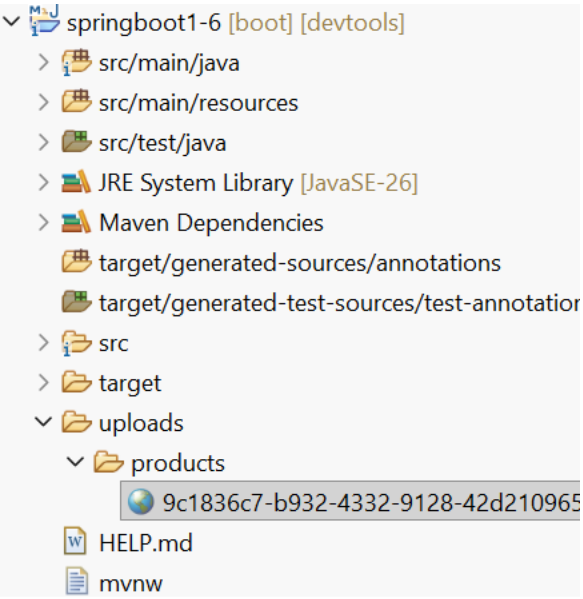
Mở rộng module Product thành:

CRUD + tìm kiếm + phân trang + upload ảnh + hiển thị ảnh, trong đó:

- Database products có thêm cột images.
- ProductDTO có MultipartFile image để nhận file upload.
- ProductMapper chuyển images giữa Entity ↔ DTO.
- Ảnh thật được lưu trong thư mục project:

uploads/products/

- SQL Server chỉ lưu **tên file/path ảnh**, không lưu binary.
- Thymeleaf hiển thị ảnh bằng /uploads/products/....
- Khi sửa sản phẩm, có thể giữ ảnh cũ nếu không upload ảnh mới



Bước 1: Sửa Entity Product

```
// Tên file ảnh
    @Column(length = 500)
    private String images;
```

Bước 2: Sửa ProductDTO

Có một điểm quan trọng:

Entity: `private String images;`

nhưng Form cần nhận: `MultipartFile image;`

Do đó DTO có cả hai:

```
// Tên file được lưu trong DB
```

```
    private String images;

    // File upload từ HTML
    private MultipartFile image;
```

Bước 3: Sửa ProductMapper

```
package vn.iotstar.mapper;

import org.springframework.stereotype.Component;

import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

@Component
public class ProductMapper {

        public ProductDTO toDTO(Product entity) {

              if (entity == null) {
                  return null;
              }

              return ProductDTO.builder()
                      .id(entity.getId())
                      .name(entity.getName())
                      .price(entity.getPrice())
                      .quantity(entity.getQuantity())
                      .description(entity.getDdescription())
                      .images(entity.getImages())
                      .build();
          }

          public Product toEntity(ProductDTO dto) {

              if (dto == null) {
                  return null;
              }

              return Product.builder()
                      .id(dto.getId())
                      .name(dto.getName())
                      .price(dto.getPrice())
                      .quantity(dto.getQuantity())
                      .description(dto.getDescription())
                      .images(dto.getImages())
                      .build();
          }

          public void updateEntity(
                  ProductDTO dto,
                  Product entity) {

              entity.setName(dto.getName());
              entity.setPrice(dto.getPrice());
              entity.setQuantity(dto.getQuantity());
              entity.setDescription(dto.getDescription());
```

```
            // Chỉ cập nhật images nếu Service
            // đã xử lý file mới
            if (dto.getImages() != null) {
                entity.setImages(dto.getImages());
            }
        }
}
```

Bước 4: Cấu hình thư mục Upload

Tạo class: configs/WebConfig.java

```
package vn.iotstar.configs;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(
            ResourceHandlerRegistry registry) {

        registry.addResourceHandler(
                "/uploads/**"
        ).addResourceLocations(
                "file:uploads/"
        );
    }
}
```

Điều này tạo mapping:

URL

│

▼

/uploads/**

│

▼

file:uploads/

Ví dụ file:

uploads/products/iphone.jpg

sẽ truy cập được:

/uploads/products/iphone.jpg

Trình duyệt:

http://localhost:8080/uploads/products/iphone.jpg

Bước 5: Cấu hình Multipart

Có nghĩa: 1 file tối đa = 5 MB; 1 request tối đa = 10 MB

```
# ================================
# FILE UPLOAD
# ================================

spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=10MB
```

Bước 6: Xử lý Upload trong ProductServiceImpl

```
package vn.iotstar.services.impl;

import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.UUID;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.multipart.MultipartFile;

import lombok.RequiredArgsConstructor;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;
import vn.iotstar.mapper.ProductMapper;
import vn.iotstar.repository.ProductRepository;
import vn.iotstar.services.ProductService;

@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService {

        private final ProductRepository productRepository;

          private final ProductMapper productMapper;

          private final Path uploadDir =
                  Paths.get("uploads/products");

          // ========================================
          // SEARCH + PAGINATION
          // ========================================

          @Override
          @Transactional(readOnly = true)
          public Page<ProductDTO> findAll(
                  String keyword,
                  int page,
                  int size) {
```

```
                Pageable pageable =
                        PageRequest.of(
                                page,
                                size,
                                Sort.by("id").descending()
                        );

                Page<Product> products;

        if (keyword == null ||
          keyword.isBlank()) {

          products =
                  productRepository
                          .findAll(pageable);

    } else {

          products =
                  productRepository
                          .findByNameContainingIgnoreCase(
                                  keyword.trim(),
                                  pageable
                          );
    }

    return products.map(
            productMapper::toDTO
    );
}

// ========================================
// FIND BY ID
// ========================================

@Override
@Transactional(readOnly = true)
public ProductDTO findById(Long id) {

    Product product =
            productRepository
                    .findById(id)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy sản phẩm"
                            )
                    );

    return productMapper.toDTO(product);
}

// ========================================
// CREATE
// ========================================

@Override
public ProductDTO create(
```

```
        ProductDTO dto) {

    try {

        String fileName =
                saveImage(dto.getImage());

        dto.setImages(fileName);

        Product product =
                productMapper.toEntity(dto);

        Product saved =
                productRepository.save(product);

        return productMapper.toDTO(saved);

    } catch (IOException e) {

        throw new RuntimeException(
                "Không thể upload ảnh",
                e
        );
    }
}

// ========================================
// UPDATE
// ========================================

@Override
public ProductDTO update(
        Long id,
        ProductDTO dto) {

    Product product =
            productRepository
                    .findById(id)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy sản phẩm"
                            )
                    );

    try {

        MultipartFile image =
                dto.getImage();

        // Có upload ảnh mới
        if (image != null &&
            !image.isEmpty()) {

            String oldImage =
                    product.getImages();

            String newImage =
                    saveImage(image);
```

```
            dto.setImages(newImage);

            // Xóa ảnh cũ
            deleteImage(oldImage);

        } else {

            // Không upload ảnh mới
            // giữ nguyên ảnh cũ
            dto.setImages(
                    product.getImages()
            );
        }

        productMapper.updateEntity(
                dto,
                product
        );

        Product updated =
                productRepository.save(product);

        return productMapper.toDTO(updated);

    } catch (IOException e) {

        throw new RuntimeException(
                "Không thể upload ảnh",
                e
        );
    }
}

// ========================================
// DELETE
// ========================================

@Override
public void delete(Long id) {

    Product product =
            productRepository
                    .findById(id)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy sản phẩm"
                            )
                    );

    // Xóa ảnh
    deleteImage(
            product.getImages()
    );

    // Xóa product
    productRepository.delete(product);
}
```

```java
// ========================================
// SAVE IMAGE
// ========================================

private String saveImage(
        MultipartFile file)
        throws IOException {

    if (file == null ||
        file.isEmpty()) {

        return null;
    }

    // Tạo thư mục
    Files.createDirectories(
            uploadDir
    );

    // Lấy extension
    String originalName =
            file.getOriginalFilename();

    String extension = "";

    if (originalName != null &&
        originalName.contains(".")) {

        extension =
                originalName.substring(
                        originalName.lastIndexOf(".")
                );
    }

    // UUID chống trùng tên
    String fileName =
            UUID.randomUUID()
                    .toString()
            + extension;

    Path target =
            uploadDir.resolve(fileName);

    try (InputStream inputStream =
                 file.getInputStream()) {

        Files.copy(
                inputStream,
                target,
                StandardCopyOption.REPLACE_EXISTING
        );
    }

    return fileName;
}

// ========================================
```

```
// DELETE IMAGE
// ========================================

private void deleteImage(
        String fileName) {

    if (fileName == null ||
        fileName.isBlank()) {

        return;
    }

    try {

        Path file =
                uploadDir.resolve(fileName);

        Files.deleteIfExists(file);

    } catch (IOException e) {

        System.err.println(
                "Không thể xóa ảnh: "
                + fileName
        );
    }
}
}
```

Bước 7: Sửa ProductController

```
@PostMapping(
            value = "/create",
            consumes = "multipart/form-data"
    )
    public String create(

            @Valid
            @ModelAttribute("product")
            ProductDTO dto,

            BindingResult result) {

        if (result.hasErrors()) {
            return "products/form";
        }

        productService.create(dto);

        return "redirect:/products";
    }
```

```
@PostMapping(
            value = "/edit/{id}",
            consumes = "multipart/form-data"
    )
    public String update(

            @PathVariable Long id,
```

```
            @Valid
            @ModelAttribute("product")
            ProductDTO dto,

            BindingResult result) {

        if (result.hasErrors()) {
            return "products/form";
        }

        productService.update(id, dto);

        return "redirect:/products";
    }
```

Còn lại các phương thức khác giữ nguyên

Bước 8: Sửa form.html, Nếu không có dòng này thì MultipartFile (enctype="multipart/form-data") sẽ không nhận được ảnh

```
<!DOCTYPE html>

<html lang="vi"

      xmlns:th="http://www.thymeleaf.org"

      xmlns:layout=
      "http://www.ultraq.net.nz/thymeleaf/layout"

      layout:decorate="~{layouts/layout}">

<head>

    <title th:text="${formTitle}">
        Product
    </title>

</head>

<body>

<section layout:fragment="content">

    <h2 th:text="${formTitle}">
        Product
    </h2>


    <form
        method="post"

        enctype="multipart/form-data"

        th:object="${product}"

        th:action="${product.id == null}
            ? @{/products/create}
            : @{/products/edit/{id}(id=${product.id})}">
```

```
<!-- ID -->

<input
    type="hidden"
    th:field="*{id}">


<!-- NAME -->

<div class="mb-3">

    <label class="form-label">
        Tên sản phẩm
    </label>

    <input
        type="text"
        class="form-control"
        th:field="*{name}">

    <div
        class="text-danger"
        th:errors="*{name}">
    </div>

</div>


<!-- PRICE -->

<div class="mb-3">

    <label class="form-label">
        Giá
    </label>

    <input
        type="number"
        step="0.01"
        class="form-control"
        th:field="*{price}">

    <div
        class="text-danger"
        th:errors="*{price}">
    </div>

</div>


<!-- QUANTITY -->

<div class="mb-3">

    <label class="form-label">
        Số lượng
```

```
            </label>

            <input
                type="number"
                class="form-control"
                th:field="*{quantity}">

            <div
                class="text-danger"
                th:errors="*{quantity}">
            </div>

        </div>


        <!-- DESCRIPTION -->

        <div class="mb-3">

            <label class="form-label">
                Mô tả
            </label>

            <textarea
                class="form-control"
                rows="4"
                th:field="*{description}">
            </textarea>

        </div>


        <!-- CURRENT IMAGE -->

        <div
            class="mb-3"
            th:if="${product.images != null
                    and !#strings.isEmpty(product.images)}">

            <label class="form-label">
                Hình ảnh hiện tại
            </label>

            <br>

            <img
                th:src="@{'/uploads/products/' + ${product.images}}"
                alt="Product Image"
                class="img-thumbnail"
                style="
                    width: 180px;
                    height: 180px;
                    object-fit: cover;
                ">

        </div>
```

```
        <!-- UPLOAD IMAGE -->

        <div class="mb-3">

            <label class="form-label">
                Chọn hình ảnh
            </label>

            <input
                type="file"
                class="form-control"
                name="image"
                accept="image/*">

            <div class="form-text">

                Định dạng: JPG, PNG, WEBP.
                Kích thước tối đa 5MB.

            </div>

        </div>


        <!-- BUTTON -->

        <button
            type="submit"
            class="btn btn-primary">

            Lưu

        </button>


        <a
            th:href="@{/products}"
            class="btn btn-secondary">

            Quay lại

        </a>

    </form>

</section>

</body>

</html>
```

Bước 9: Hiển thị ảnh trong list.html

```
<!DOCTYPE html>

<html lang="vi" xmlns:th="https://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/layout}">
```

```
<head>
<title>Danh sách sản phẩm</title>
</head>
<body>
	<section layout:fragment="content">
		<div class="d-flex justify-content-between">
			<h2>Danh sách sản phẩm</h2>
			<a class="btn btn-primary" th:href="@{/products/create}"> + Thêm
				sản phẩm </a>
		</div>

		<!-- SEARCH -->
		<form class="row mt-3 mb-3" method="get" th:action="@{/products}">
			<div class="col-md-6">
				<input type="text" class="form-control" name="keyword"
					th:value="${keyword}" placeholder="Nhập tên sản phẩm...">
			</div>
			<div class="col-md-2">
				<select class="form-select" name="size">
					<option value="5" th:selected="${size == 5}">5</option>
					<option value="10" th:selected="${size == 10}">10</option>
					<option value="20" th:selected="${size == 20}">20</option>
				</select>
			</div>
			<div class="col-md-2">
				<button class="btn btn-success" type="submit">Tìm kiếm</button>
			</div>
		</form>

		<!-- TABLE -->
		<table class="table table-bordered table-hover">
```

```
                <thead class="table-dark">
                        <tr>
                                <th>ID</th>
                                <th>Hình ảnh</th>
                                <th>Tên sản phẩm</th>
                                <th>Giá</th>
                                <th>Số lượng</th>
                                <th>Mô tả</th>
                                <th>Thao tác</th>
                        </tr>
                </thead>

                <tbody>
                        <tr th:each="product : ${products.content}">
                                <td th:text="${product.id}"></td>
                                <td><img
                                        th:if="${product.images != null
            and !#strings.isEmpty(product.images)}"
                                        th:src="@{'/uploads/products/' +
${product.images}}"
                                        alt="Product Image"
                                        style="width: 80px; height: 80px; object-fit:
cover; border-radius: 8px;">

                                        <span
                                        th:if="${product.images == null
            or #strings.isEmpty(product.images)}">

                                                Chưa có ảnh </span></td>

                                <td th:text="${product.name}"></td>

                                <td><span
                                        th:text="${#numbers.formatDecimal(
                    product.price, 0, 'COMMA', 2, 'POINT'
                )}">
                                </span></td>

                                <td th:text="${product.quantity}"></td>

                                <td th:text="${product.description}"></td>

                                <td><a class="btn btn-warning btn-sm"
```

```
                                    th:href="@{/products/edit/{id}(id=${product.id})}">
Sửa </a> <a
                                    class="btn btn-danger btn-sm"
       th:href="@{/products/delete/{id}(id=${product.id})}"
                                    onclick="return confirm('Bạn có chắc muốn xóa?')">
Xóa </a></td>

                        </tr>

                    </tbody>

                </table>


                <!-- EMPTY -->

                <div class="alert alert-warning"
                    th:if="${products.totalElements == 0}">Không tìm thấy sản phẩm.

                </div>


                <!-- PAGINATION -->

                <nav th:if="${products.totalPages > 1}">

                    <ul class="pagination">

                        <!-- Previous -->

                        <li class="page-item"
                            th:classappend="${products.first} ? 'disabled'"><a
                            class="page-link"
                            th:href="@{/products(
                        page=${products.number - 1},
                        size=${size},
                        keyword=${keyword}
                    )}">

                                Previous </a></li>


                        <!-- Page number -->

                        <li class="page-item"
                            th:each="i : ${#numbers.sequence(
                        0,
                        products.totalPages - 1
                    )}"
                            th:classappend="${i == products.number}
                        ? 'active'">

                            <a class="page-link" th:text="${i + 1}"
                            th:href="@{/products(
                        page=${i},
                        size=${size},
                        keyword=${keyword}
```

```
                    )}">

                        </a>

                        </li>


                        <!-- Next -->

                        <li class="page-item"
                                th:classappend="${products.last}
                    ? 'disabled'">

                                <a class="page-link"
                                th:href="@{/products(
                            page=${products.number + 1},
                            size=${size},
                            keyword=${keyword}
                    )}">

                                                Next </a>

                        </li>

                    </ul>

                </nav>

        </section>

</body>

</html>
```

Ví dụ 3: Ta nâng cấp thiết kế từ:

Product.images = "abc.jpg"

thành quan hệ chuẩn:

Product

│

│ 1

│

│ N

▼

ProductImage

Thiết kế này phù hợp hơn cho **website thương mại điện tử**, vì một sản phẩm có thể có nhiều hình ảnh: ảnh chính, ảnh mặt trước, mặt sau, chi tiết, thumbnail...

Bước 1: sửa lại Entity Product

Bây giờ quan hệ Java là:

Product

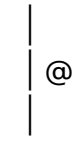
│

│ @OneToMany



List<ProductImage>

và:



Đây là quan hệ hai chiều:

1

Product ——————————< ProductImage

N

```java
package vn.iotstar.entity;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OrderBy;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200, columnDefinition = "NVARCHAR(1000)")
    private String name;
```

```
    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal price;

    @Column(nullable = false)
    private Integer quantity;

    @Column(length = 1000, columnDefinition = "NVARCHAR(1000)")
    private String description;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

 // Tên file ảnh
    //@Column(length = 500)
   // private String images;
    // ====================================
    // PRODUCT 1 ---- N PRODUCT IMAGE
    // ====================================

    @OneToMany(
        mappedBy = "product",
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    @OrderBy("displayOrder ASC")
    @Builder.Default
    private List<ProductImage> images =
            new ArrayList<>();

}
====================================
```

ProductImage.java

```
package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "product_images")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductImage {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // ====================================
    // N PRODUCT IMAGE ---- 1 PRODUCT
    // ====================================
```

```
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(
        name = "product_id",
        nullable = false
    )
    private Product product;

    @Column(
        name = "image_url",
        nullable = false,
        length = 500
    )
    private String imageUrl;

    @Column(name = "is_primary")
    @Builder.Default
    private Boolean primary = false;

    @Column(name = "display_order")
    @Builder.Default
    private Integer displayOrder = 0;

    @Column(name = "created_at")
    private LocalDateTime createdAt;
}
==============================
```

Bước 2: Tạo ProductImageDTO

```
package vn.iotstar.dto;

import lombok.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductImageDTO {

    private Long id;

    private String imageUrl;

    private Boolean primary;

    private Integer displayOrder;
}
```

Sửa lại ProductDTO.java

```
package vn.iotstar.dto;

import java.math.BigDecimal;
import java.util.List;
```

```java
import org.springframework.web.multipart.MultipartFile;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ProductDTO {

    private Long id;

    @NotBlank(
        message = "Tên sản phẩm không được để trống"
    )
    @Size(max = 200)
    private String name;

    @NotNull(
        message = "Giá không được để trống"
    )
    @DecimalMin(value = "0")
    private BigDecimal price;

    @NotNull(
        message = "Số lượng không được để trống"
    )
    @Min(value = 0)
    private Integer quantity;

    @Size(max = 1000)
    private String description;

    // ==================================
    // DANH SÁCH ẢNH ĐÃ LƯU
    // ==================================

    private List<ProductImageDTO> images;

    // ==================================
    // CÁC FILE ĐƯỢC UPLOAD
    // ==================================

    private List<MultipartFile> imageFiles;
}
```
==========

Bước 3: Tạo ProductImageMapper

```
package vn.iotstar.mapper;


import org.springframework.stereotype.Component;

import vn.iotstar.dto.ProductImageDTO;
import vn.iotstar.entity.ProductImage;

@Component
public class ProductImageMapper {

    public ProductImageDTO toDTO(
            ProductImage entity) {

        if (entity == null) {
            return null;
        }

        return ProductImageDTO.builder()
                .id(entity.getId())
                .imageUrl(entity.getImageUrl())
                .primary(entity.getPrimary())
                .displayOrder(
                    entity.getDisplayOrder()
                )
                .build();
    }
}
=======================
```

Sửa lại ProductMapper sử dụng ProductImageMapper

```
package vn.iotstar.mapper;

import org.springframework.stereotype.Component;

import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

import lombok.RequiredArgsConstructor;


import java.util.stream.Collectors;

@Component
@RequiredArgsConstructor
public class ProductMapper {

    private final ProductImageMapper imageMapper;

    // =====================================
    // ENTITY → DTO
    // =====================================

    public ProductDTO toDTO(Product entity) {
```

```
    if (entity == null) {
        return null;
    }

    return ProductDTO.builder()
            .id(entity.getId())
            .name(entity.getName())
            .price(entity.getPrice())
            .quantity(entity.getQuantity())
            .description(entity.getDescription())

            .images(
                entity.getImages()
                    .stream()
                    .map(imageMapper::toDTO)
                    .collect(
                        Collectors.toList()
                    )
            )

            .build();
}

// ====================================
// DTO → ENTITY
// ====================================

public Product toEntity(
        ProductDTO dto) {

    if (dto == null) {
        return null;
    }

    return Product.builder()
            .id(dto.getId())
            .name(dto.getName())
            .price(dto.getPrice())
            .quantity(dto.getQuantity())
            .description(dto.getDescription())
            .build();
}

// ====================================
// UPDATE
// ====================================

public void updateEntity(
        ProductDTO dto,
        Product entity) {

    entity.setName(dto.getName());
    entity.setPrice(dto.getPrice());
    entity.setQuantity(dto.getQuantity());
    entity.setDescription(
            dto.getDescription()
    );
}
```

```
}
======================
```

Bước 4: Tạo ProductImageRepository

```
package vn.iotstar.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import vn.iotstar.entity.ProductImage;

import java.util.List;

public interface ProductImageRepository
        extends JpaRepository<ProductImage, Long> {


    List<ProductImage>
    findByProductIdOrderByDisplayOrderAsc(
            Long productId
    );


    long countByProductId(Long productId);
}
================================================

package vn.iotstar.repository;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

import vn.iotstar.entity.Product;

public interface ProductRepository extends JpaRepository<Product, Long> {


      Page<Product> findByNameContainingIgnoreCase(String name, Pageable pageable);
}
```

Bước 5: ProductService

Bước 8: **Service xử lý nhiều ảnh**

```
package vn.iotstar.services;

import java.io.IOException;
import java.util.List;

import org.springframework.data.domain.Page;
import org.springframework.web.multipart.MultipartFile;

import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

public interface ProductService {
```

```
    Page<ProductDTO> findAll(
            String keyword,
            int page,
            int size
    );

    ProductDTO findById(Long id);

    ProductDTO create(ProductDTO dto);

    ProductDTO update(
            Long id,
            ProductDTO dto
    );

    void delete(Long id);

    String saveImage(
            MultipartFile file
    ) throws IOException;

    void saveImages(
            Product product,
            List<MultipartFile> files
    ) throws IOException;

    Long deleteImage(Long imageId);

    void deleteFile(String fileName);
}
================================================
```

Bước 6: ProductServiceImpl.java

```
package vn.iotstar.services.impl;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.time.LocalDateTime;
import java.util.List;
import java.util.UUID;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.multipart.MultipartFile;

import lombok.RequiredArgsConstructor;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;
import vn.iotstar.entity.ProductImage;
```

```java
import vn.iotstar.mapper.ProductMapper;
import vn.iotstar.repository.ProductImageRepository;
import vn.iotstar.repository.ProductRepository;
import vn.iotstar.services.ProductService;

@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService {

    private final ProductRepository productRepository;

    private final ProductMapper productMapper;

    private final ProductImageRepository imageRepository;

    private final Path uploadDir =
            Paths.get("uploads/products");


    // ======================================================
    // SEARCH + PAGINATION
    // ======================================================

    @Override
    @Transactional(readOnly = true)
    public Page<ProductDTO> findAll(
            String keyword,
            int page,
            int size) {

        Pageable pageable =
                PageRequest.of(
                        page,
                        size,
                        Sort.by("id").descending()
                );

        Page<Product> products;

        if (keyword == null ||
            keyword.isBlank()) {

            products =
                    productRepository
                            .findAll(pageable);

        } else {

            products =
                    productRepository
                            .findByNameContainingIgnoreCase(
                                    keyword.trim(),
                                    pageable
                            );
        }

        return products.map(
```

```
            productMapper::toDTO
    );
}


// ====================================================
// FIND BY ID
// ====================================================

@Override
@Transactional(readOnly = true)
public ProductDTO findById(Long id) {

    Product product =
            productRepository
                    .findById(id)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy sản phẩm"
                            )
                    );

    return productMapper.toDTO(product);
}


// ====================================================
// CREATE
// ====================================================

@Override
public ProductDTO create(
        ProductDTO dto) {

    try {

        // DTO -> Entity
        Product product =
                productMapper.toEntity(dto);

        // Lưu Product trước
        Product saved =
                productRepository.save(product);

        // Upload nhiều ảnh
        saveImages(
                saved,
                dto.getImageFiles()
        );

        // Lưu quan hệ ProductImage
        Product result =
                productRepository.save(saved);

        // Entity -> DTO
        return productMapper.toDTO(result);

    } catch (IOException e) {
```

```
            throw new RuntimeException(
                    "Không thể upload hình ảnh",
                    e
            );
        }
    }


    // ====================================================
    // UPDATE
    // ====================================================

    @Override
    public ProductDTO update(
            Long id,
            ProductDTO dto) {

        try {

            Product product =
                    productRepository
                            .findById(id)
                            .orElseThrow(() ->
                                    new RuntimeException(
                                            "Không tìm thấy sản phẩm"
                                    )
                            );

            // Update Product
            productMapper.updateEntity(
                    dto,
                    product
            );

            // Upload ảnh mới
            saveImages(
                    product,
                    dto.getImageFiles()
            );

            Product updated =
                    productRepository.save(product);

            return productMapper.toDTO(updated);

        } catch (IOException e) {

            throw new RuntimeException(
                    "Không thể upload hình ảnh",
                    e
            );
        }
    }


    // ====================================================
    // SAVE ONE IMAGE
```

```java
// ====================================================

@Override
public String saveImage(
        MultipartFile file)
        throws IOException {

    if (file == null ||
        file.isEmpty()) {

        return null;
    }

    // Tạo thư mục nếu chưa tồn tại
    Files.createDirectories(
            uploadDir
    );

    String originalName =
            file.getOriginalFilename();

    String extension = "";

    if (originalName != null &&
        originalName.contains(".")) {

        extension =
                originalName.substring(
                        originalName.lastIndexOf(".")
                );
    }

    // Tạo tên file duy nhất
    String fileName =
            UUID.randomUUID()
                    .toString()
            + extension;

    Path target =
            uploadDir.resolve(fileName);

    Files.copy(
            file.getInputStream(),
            target,
            StandardCopyOption.REPLACE_EXISTING
    );

    return fileName;
}


// ====================================================
// SAVE MULTIPLE IMAGES
// ====================================================

@Override
public void saveImages(
        Product product,
```

```
        List<MultipartFile> files
) throws IOException {

    if (files == null || files.isEmpty()) {
        return;
    }

    int currentOrder =
            product.getImages().size();

    for (MultipartFile file : files) {

        if (file == null || file.isEmpty()) {
            continue;
        }

        String fileName = saveImage(file);

        boolean isPrimary =
                product.getImages().isEmpty();

        ProductImage image =
                ProductImage.builder()
                        .product(product)
                        .imageUrl(fileName)
                        .primary(isPrimary)
                        .displayOrder(currentOrder++)
                        .createdAt(LocalDateTime.now())
                        .build();

        product.getImages().add(image);
    }
}


// ======================================================
// DELETE PRODUCT
// ======================================================

@Override
public void delete(Long id) {

    Product product =
            productRepository
                    .findById(id)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy sản phẩm"
                            )
                    );

    // ----------------------------------------------
    // 1. Xóa file vật lý
    // ----------------------------------------------

    if (product.getImages() != null) {

        for (ProductImage image :
```

```
                product.getImages()) {

            deleteFile(
                    image.getImageUrl()
            );
        }
    }

    // ------------------------------------------
    // 2. Xóa Product
    // ------------------------------------------
    //
    // CascadeType.ALL
    // orphanRemoval = true
    //
    // => ProductImage cũng bị xóa
    //
    productRepository.delete(product);
}


// ======================================================
// DELETE ONE IMAGE
// ======================================================

@Override
public Long deleteImage(Long imageId) {

    ProductImage image =
            imageRepository
                    .findById(imageId)
                    .orElseThrow(() ->
                            new RuntimeException(
                                    "Không tìm thấy hình ảnh"
                            )
                    );

    Product product =
            image.getProduct();

    Long productId =
            product.getId();

    boolean wasPrimary =
            Boolean.TRUE.equals(
                    image.getPrimary()
            );

    // Xóa file vật lý
    deleteFile(
            image.getImageUrl()
    );

    // Xóa khỏi collection
    product.getImages()
           .remove(image);

    // Xóa DB
```

```
        imageRepository.delete(image);

        // Nếu xóa ảnh chính
        // thì chọn ảnh đầu tiên còn lại
        if (wasPrimary &&
            !product.getImages().isEmpty()) {

            ProductImage newPrimary =
                    product.getImages().get(0);

            newPrimary.setPrimary(true);
            newPrimary.setDisplayOrder(0);
        }

        return productId;
    }


    // =====================================================
    // DELETE PHYSICAL FILE
    // =====================================================

    @Override
    public void deleteFile(String fileName) {

        if (fileName == null || fileName.isBlank()) {
            return;
        }

        try {
            Path uploadRoot = uploadDir
                    .toAbsolutePath()
                    .normalize();

            Path file = uploadRoot
                    .resolve(fileName)
                    .normalize();

            // Chống Path Traversal
            if (!file.startsWith(uploadRoot)) {
                throw new SecurityException(
                        "Tên file không hợp lệ"
                );
            }

            Files.deleteIfExists(file);

        } catch (IOException e) {

            System.err.println(
                    "Không thể xóa file: " + fileName
            );
        }
    }
}
=========================
```

Bước 7: ProductController.java

```
package vn.iotstar.controllers;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;

import vn.iotstar.dto.ProductDTO;
import vn.iotstar.services.ProductService;


import org.springframework.data.domain.Page;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;


    // =====================================================
    // LIST + SEARCH + PAGINATION
    // =====================================================

    @GetMapping
    public String listProducts(

            @RequestParam(defaultValue = "")
            String keyword,

            @RequestParam(defaultValue = "0")
            int page,

            @RequestParam(defaultValue = "5")
            int size,

            Model model) {

        Page<ProductDTO> productPage =
                productService.findAll(
                        keyword,
                        page,
                        size
                );

        model.addAttribute(
                "products",
                productPage
        );

        model.addAttribute(
```

```
            "keyword",
            keyword
    );

    model.addAttribute(
            "size",
            size
    );

    return "products/list";
}


// ======================================================
// CREATE FORM
// ======================================================

@GetMapping("/create")
public String createForm(
        Model model) {

    model.addAttribute(
            "product",
            new ProductDTO()
    );

    model.addAttribute(
            "formTitle",
            "Thêm sản phẩm"
    );

    return "products/form";
}


// ======================================================
// CREATE
// ======================================================

@PostMapping(
        value = "/create",
        consumes = "multipart/form-data"
)
public String create(

        @Valid
        @ModelAttribute("product")
        ProductDTO dto,

        BindingResult result,

        Model model,

        RedirectAttributes redirectAttributes) {

    // Validation
    if (result.hasErrors()) {
```

```
            model.addAttribute(
                    "formTitle",
                    "Thêm sản phẩm"
            );

            return "products/form";
        }

        try {

            productService.create(dto);

            redirectAttributes.addFlashAttribute(
                    "success",
                    "Thêm sản phẩm thành công"
            );

            return "redirect:/products";

        } catch (Exception e) {

            model.addAttribute(
                    "formTitle",
                    "Thêm sản phẩm"
            );

            model.addAttribute(
                    "error",
                    "Không thể thêm sản phẩm: "
                            + e.getMessage()
            );

            return "products/form";
        }
    }


    // ======================================================
    // EDIT FORM
    // ======================================================

    @GetMapping("/edit/{id}")
    public String editForm(

            @PathVariable Long id,

            Model model) {

        ProductDTO product =
                productService.findById(id);

        model.addAttribute(
                "product",
                product
        );

        model.addAttribute(
                "formTitle",
```

```
            "Cập nhật sản phẩm"
        );

        return "products/form";
    }


    // ====================================================
    // UPDATE
    // ====================================================

    @PostMapping(
            value = "/edit/{id}",
            consumes = "multipart/form-data"
    )
    public String update(

            @PathVariable Long id,

            @Valid
            @ModelAttribute("product")
            ProductDTO dto,

            BindingResult result,

            Model model,

            RedirectAttributes redirectAttributes) {

        // Validation
        if (result.hasErrors()) {

            model.addAttribute(
                    "formTitle",
                    "Cập nhật sản phẩm"
            );

            return "products/form";
        }

        try {

            productService.update(
                    id,
                    dto
            );

            redirectAttributes.addFlashAttribute(
                    "success",
                    "Cập nhật sản phẩm thành công"
            );

            return "redirect:/products";

        } catch (Exception e) {

            model.addAttribute(
                    "formTitle",
```

```
                "Cập nhật sản phẩm"
        );

        model.addAttribute(
                "error",
                "Không thể cập nhật sản phẩm: "
                        + e.getMessage()
        );

        return "products/form";
    }
}


// ======================================================
// DELETE PRODUCT
// ======================================================

@GetMapping("/delete/{id}")
public String delete(

        @PathVariable Long id,

        RedirectAttributes redirectAttributes) {

    try {

        productService.delete(id);

        redirectAttributes.addFlashAttribute(
                "success",
                "Xóa sản phẩm thành công"
        );

    } catch (Exception e) {

        redirectAttributes.addFlashAttribute(
                "error",
                "Không thể xóa sản phẩm"
        );
    }

    return "redirect:/products";
}


// ======================================================
// DELETE PRODUCT IMAGE
// ======================================================

@PostMapping("/image/delete/{imageId}")
public String deleteImage(

        @PathVariable Long imageId,

        RedirectAttributes redirectAttributes) {

    try {
```

```
            /*
             * Service xóa:
             *
             * 1. File vật lý
             * 2. ProductImage trong DB
             * 3. Nếu ảnh chính -> chọn ảnh khác
             *
             * Sau đó trả về productId
             */
            Long productId =
                    productService.deleteImage(
                            imageId
                    );

            redirectAttributes.addFlashAttribute(
                    "success",
                    "Xóa hình ảnh thành công"
            );

            // Quay lại đúng form Edit Product
            return "redirect:/products/edit/"
                    + productId;

        } catch (Exception e) {

            redirectAttributes.addFlashAttribute(
                    "error",
                    "Không thể xóa hình ảnh: "
                            + e.getMessage()
            );

            return "redirect:/products";
        }
    }

}
```

Bước 8: Chỉnh lại list.html

```
<!DOCTYPE html>

<html lang="vi" xmlns:th="https://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/layout}">

<head>

<title>Danh sách sản phẩm</title>

</head>

<body>

      <section layout:fragment="content">

            <div class="d-flex justify-content-between">
```

```
        <h2>Danh sách sản phẩm</h2>

        <a class="btn btn-primary" th:href="@{/products/create}"> + Thêm
            sản phẩm </a>

</div>


<!-- SEARCH -->

<form class="row mt-3 mb-3" method="get" th:action="@{/products}">

        <div class="col-md-6">

            <input type="text" class="form-control" name="keyword"
                th:value="${keyword}" placeholder="Nhập tên sản phẩm...">

        </div>

        <div class="col-md-2">

            <select class="form-select" name="size">

                <option value="5" th:selected="${size == 5}">5</option>

                <option value="10" th:selected="${size == 10}">10</option>

                <option value="20" th:selected="${size == 20}">20</option>

            </select>

        </div>

        <div class="col-md-2">

            <button class="btn btn-success" type="submit">Tìm kiếm</button>

        </div>

</form>


<!-- TABLE -->

<table class="table table-bordered table-hover">

        <thead class="table-dark">

            <tr>

                <th>ID</th>

                <th>Hình ảnh</th>

                <th>Tên sản phẩm</th>

                <th>Giá</th>
```

```
                        <th>Số lượng</th>

                        <th>Mô tả</th>

                        <th>Thao tác</th>

                    </tr>

                </thead>


                <tbody>

                    <tr th:each="product : ${products.content}">

                        <td th:text="${product.id}"></td>


                        <td>

                            <div class="d-flex gap-2 flex-wrap"
                                th:if="${product.images != null}">


                                <img th:each="image : ${product.images}"
th:if="${image.primary}"
                                    th:src="@{'/uploads/products/' +
${image.imageUrl}}"

                                    th:alt="${product.name}"
                                    style="width: 70px; height: 70px;
object-fit: cover; border-radius: 8px;">

                            </div> <span
                            th:if="${product.images == null
            or #strings.isEmpty(product.images)}">

                                Chưa có ảnh </span>
                        </td>




                        <td th:text="${product.name}"></td>

                        <td><span
                                th:text="${#numbers.formatDecimal(
                    product.price, 0, 'COMMA', 2, 'POINT'
                )}">
                        </span></td>

                        <td th:text="${product.quantity}"></td>

                        <td th:text="${product.description}"></td>

                        <td><a class="btn btn-warning btn-sm"
                            th:href="@{/products/edit/{id}(id=${product.id})}">
Sửa </a> <a
                            class="btn btn-danger btn-sm"
```

```
        th:href="@{/products/delete/{id}(id=${product.id})}"
                                    onclick="return confirm('Bạn có chắc muốn xóa?')">
Xóa </a></td>

                        </tr>

                    </tbody>

                </table>


                <!-- EMPTY -->

                <div class="alert alert-warning"
                    th:if="${products.totalElements == 0}">Không tìm thấy sản phẩm.

                </div>


                <!-- PAGINATION -->

                <nav th:if="${products.totalPages > 1}">

                    <ul class="pagination">

                        <!-- Previous -->

                        <li class="page-item"
                            th:classappend="${products.first} ? 'disabled'"><a
                            class="page-link"
                            th:href="@{/products(
                        page=${products.number - 1},
                        size=${size},
                        keyword=${keyword}
                    )}">

                                Previous </a></li>


                        <!-- Page number -->

                        <li class="page-item"
                            th:each="i : ${#numbers.sequence(
                        0,
                        products.totalPages - 1
                    )}"
                            th:classappend="${i == products.number}
                        ? 'active'">

                            <a class="page-link" th:text="${i + 1}"
                            th:href="@{/products(
                        page=${i},
                        size=${size},
                        keyword=${keyword}
                    )}">

                        </a>
```

```
                        </li>


                        <!-- Next -->

                        <li class="page-item"
                              th:classappend="${products.last}
                    ? 'disabled'">

                              <a class="page-link"
                              th:href="@{/products(
                        page=${products.number + 1},
                        size=${size},
                        keyword=${keyword}
                    )}">

                                    Next </a>

                        </li>

                    </ul>

            </nav>

      </section>

</body>

</html>
```

Bước 9: Chỉnh lại form.html

```
<!DOCTYPE html>
<html lang="vi" xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/layout.html}">

<head>

<title th:text="${formTitle}">Product</title>

</head>


<body>

      <section layout:fragment="content">

            <!-- ============================================== -->
            <!-- TITLE -->
            <!-- ============================================== -->

            <h2 th:text="${formTitle}">Product</h2>


            <!-- ============================================== -->
```

```
            <!-- SUCCESS MESSAGE -->
            <!-- ================================================ -->

            <div th:if="${success}" class="alert alert-success"
                  th:text="${success}"></div>


            <!-- ================================================ -->
            <!-- ERROR MESSAGE -->
            <!-- ================================================ -->

            <div th:if="${error}" class="alert alert-danger" th:text="${error}">
            </div>


            <!-- ================================================ -->
            <!-- PRODUCT FORM -->
            <!-- ================================================ -->

            <form method="post" enctype="multipart/form-data"
                  th:object="${product}"
                  th:action="${product.id == null}
                  ? @{/products/create}
                  : @{/products/edit/{id}(id=${product.id})}">

                  <input type="hidden" th:field="*{id}">

                  <div class="mb-3">

                        <label class="form-label"> Tên sản phẩm </label> <input
type="text"
                              class="form-control" th:field="*{name}">

                        <div class="text-danger" th:errors="*{name}"></div>

                  </div>


                  <div class="mb-3">

                        <label class="form-label"> Giá </label> <input type="number"
                              step="0.01" class="form-control" th:field="*{price}">

                        <div class="text-danger" th:errors="*{price}"></div>

                  </div>


                  <div class="mb-3">

                        <label class="form-label"> Số lượng </label> <input
type="number"
                              class="form-control" th:field="*{quantity}">

                        <div class="text-danger" th:errors="*{quantity}"></div>

                  </div>
```

```
                    <div class="mb-3">

                        <label class="form-label"> Mô tả </label>

                        <textarea class="form-control" rows="4"
th:field="*{description}">
        </textarea>

                    </div>


                    <!-- CURRENT IMAGES -->

                    <div class="mb-4"
                        th:if="${product.images != null
                and !product.images.isEmpty()}">

                        <label class="form-label fw-bold"> Hình ảnh hiện tại </label>

                        <div class="d-flex gap-3 flex-wrap">

                            <div th:each="image : ${product.images}" class="card"
                                style="width: 150px;">

                                <img
                                    th:src="@{'/uploads/products/'
                        + ${image.imageUrl}}"
                                    th:alt="${product.name}" class="card-img-top"
                                    style="width: 150px; height: 150px; object-
fit: cover;">

                                <div class="card-body text-center p-2">

                                    <div th:if="${image.primary}" class="mb-2">

                                        <span class="badge bg-success"> ★ Ảnh
chính </span>

                                    </div>

                                    <button type="button" class="btn btn-danger
btn-sm w-100"
                                        th:data-
url="@{'/products/image/delete/' + ${image.id}}"
                                        onclick="deleteImage(this)">🗑 Xóa
ảnh</button>

                                </div>

                            </div>

                        </div>

                    </div>
```

```
                    <!-- UPLOAD -->
                    <div class="mb-3">
                        <label class="form-label fw-bold"> Thêm hình ảnh mới </label>
<input
                            type="file" class="form-control" name="imageFiles"
multiple
                            accept="image/*">
                        <div class="form-text">Có thể chọn nhiều hình ảnh.</div>
                    </div>

                    <button type="submit" class="btn btn-primary">💾 Lưu</button>
                    <a th:href="@{/products}" class="btn btn-secondary"> Quay lại </a>
                </form>
                <script>
function deleteImage(button) {
    if (!confirm(
            "Bạn có chắc muốn xóa hình ảnh này?"
    )) {
        return;
    }
    const url =
        button.getAttribute("data-url");
    fetch(url, {
        method: "POST",
        headers: {
            "X-Requested-With": "XMLHttpRequest"
        }
    })
    .then(response => {
        if (response.ok) {
            window.location.reload();
        } else {
            alert(
                "Không thể xóa hình ảnh!"
            );
        }
    })
    .catch(error => {
```

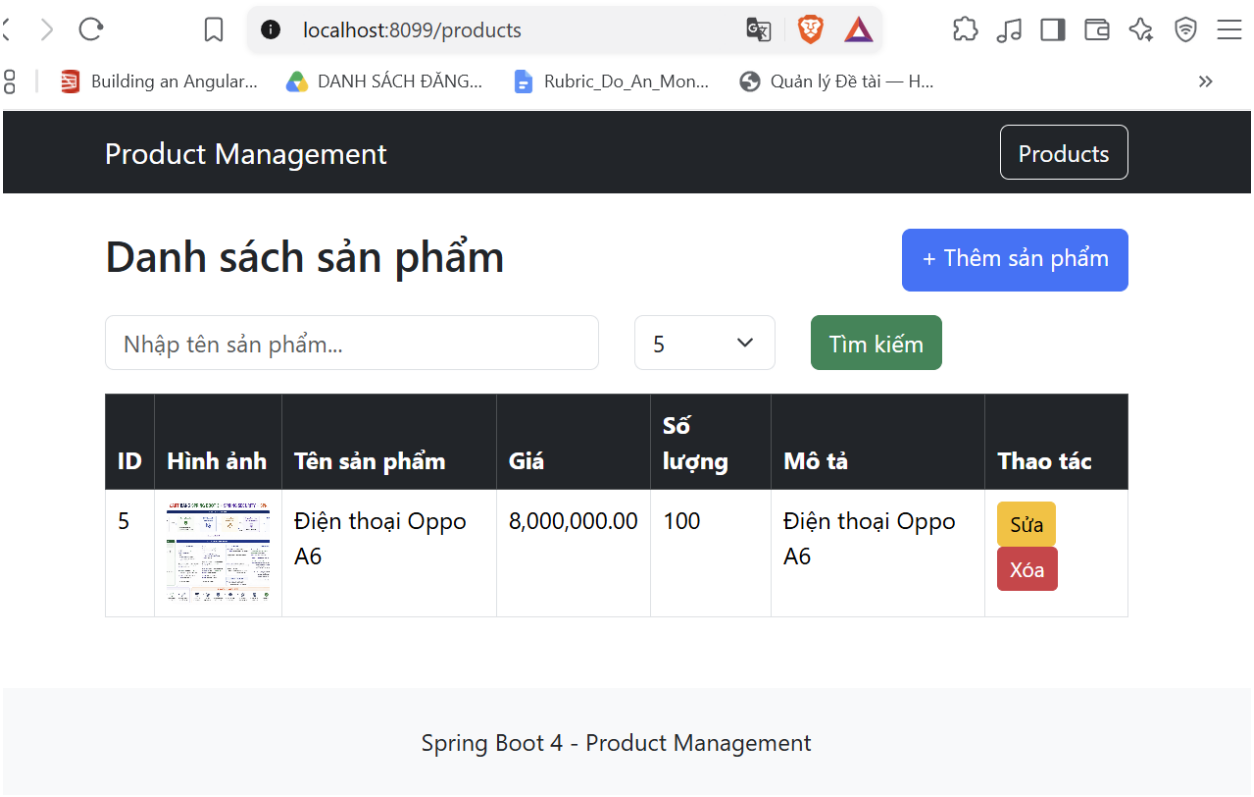
```
            console.error(error);

            alert(
                "Có lỗi xảy ra khi xóa hình ảnh!"
            );
        });
}

</script>
      </section>

</body>

</html>
```



Cập nhật sản phẩm

Tên sản phẩm

Điện thoại Oppo A6

Giá

8000000.00

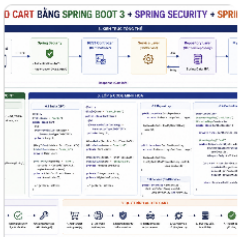
Số lượng

100

Mô tả

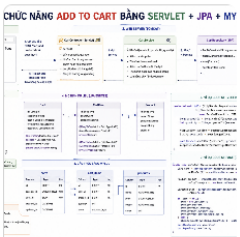
Điện thoại Oppo A6

Hình ảnh hiện tại



★ Ảnh chính

Xóa ảnh



Xóa ảnh

Thêm hình ảnh mới

Choose Files No file chosen

Có thể chọn nhiều hình ảnh.

Lưu Quay lại